STAC WARS · PS2

SOLAR RADIATION ANALYSIS AND PREDICTION

Complete Code, Results, Visualizations & Run Instructions

Dataset: olarPrediction.csv

Project scope: Part A — Data Visualization; Part B — Drawing Inferences; Part C — Solar Day Analysis; Bonus — Solar Radiation Prediction.

Prepared using the provided PS2 problem statement and dataset. The script generates the plots, daily report, feature-importance chart, and machine-learning evaluation metrics.

1. Executive Summary

This report analyzes the supplied solar-observation dataset, creates time-series and environmental visualizations, summarizes daily solar activity, and trains a Random Forest regression model to estimate solar radiation. Numerical findings below are calculated by running the included Python script against the supplied CSV.

PS2 SOLAR RADIATION ANALYSIS — SUMMARY

Valid observations: 32,686

Date range: 2016-09-01 00:00:08 to 2016-12-31 23:55:01

Mean radiation: 207.1247

Maximum radiation: 1601.2600

Mean-radiation peak hour: 12:00

Strongest absolute linear correlation: Temperature (0.7350)

Temperature correlation: 0.7350

Humidity correlation: -0.2262

Global peak date/time: 2016-09-04 / 2016-09-04 12:15:04

Peak radiation: 1601.2600

Temperature at peak: 61

Humidity at peak: 93

Wind speed at peak: 9.0

Sunrise: 2016-09-04 06:08:00; sunset: 2016-09-04 18:35:00; peak after sunrise: 0 days 06:07:04

Random Forest RMSE: 82.5681

Random Forest R2: 0.9314

Top 5 days by average radiation:

Date DailyMaxRadiation DailyAverageRadiation ObservationCount

2016-09-19 1085.75 335.504400 275

2016-09-02 1062.01 333.471219 279

2016-09-22 1053.17 325.515880 284

2016-09-25 1075.29 319.921514 284

2016-09-28 1029.30 312.400390 282

Top model features:

TimeCos 0.757460

Humidity 0.111629

Temperature 0.029523

DaySin 0.017985

DayCos 0.015610

TimeSin 0.015305

WindDirection(Degrees) 0.013499

Pressure 0.013292

SunsetMinutes 0.010849

SunriseMinutes 0.008197

Important interpretation note: the dataset column named **Data** contains a date-like timestamp, while **Time** contains the observation clock time. The script combines the calendar date from Data with Time to create DateTime. Radiation is labelled in the dataset's recorded units; do not assume SI units without dataset documentation.

2. Required Dependencies

Package	Purpose
pandas	Read CSV, parse timestamps, group daily observations
numpy	Numerical calculations and cyclical time features
matplotlib	Create and save plots
seaborn	Scatter plots, correlation matrix, and heatmap
scikit-learn	Train/test split, Random Forest, RMSE and R^2

Install in terminal / command prompt:

```
python -m pip install pandas numpy matplotlib seaborn scikit-learn
```

Python 3.10 or newer is recommended. A virtual environment is optional but helps keep dependencies isolated.

3. How to Run the Code

1. Download or copy **solar_ps2_analysis.py** and **olarPrediction.csv** into the same folder.

2. Open a terminal or command prompt in that folder.

3. Install the packages using the command above.

4. Run **python solar_ps2_analysis.py** (on some systems use **python3 solar_ps2_analysis.py**).

5. After the run, open the **solar_ps2_outputs** folder. It contains the PNG plots, daily CSV summary, feature-importance CSV, and text summary.

Google Colab: upload both files to the Colab session, then run **!pip install pandas numpy matplotlib seaborn scikit-learn**, followed by **!python solar_ps2_analysis.py** after uploading the script. If using a different folder, update DATA_FILE in the script.

4. Analysis Method and Visualizations

Plot 1 — Radiation over time

The script resamples radiation to hourly means before plotting. This makes the overall daily/seasonal pattern easier to read than plotting every raw observation.

Plot 2 — Radiation vs temperature and humidity

Scatter plots show whether radiation tends to rise or fall as temperature and humidity change. Scatter does not prove causation.

Plot 3 — Correlation heatmap

Pearson correlations summarize linear relationships among radiation and the environmental variables. Correlation is not causation and can miss nonlinear relationships.

Bonus plot — Date × hour heatmap

Each cell shows mean radiation for a date and clock hour. It can reveal recurring daylight patterns, low-radiation periods, and differences between days.

Bonus ML — Random Forest regression

The model uses weather measurements plus cyclical time-of-day and day-of-year features. An 80/20 random train/test split is used; RMSE measures typical error magnitude, and R^2 measures variance explained relative to a constant-mean baseline.

Figure 1. Hourly mean radiation over the full observation period.

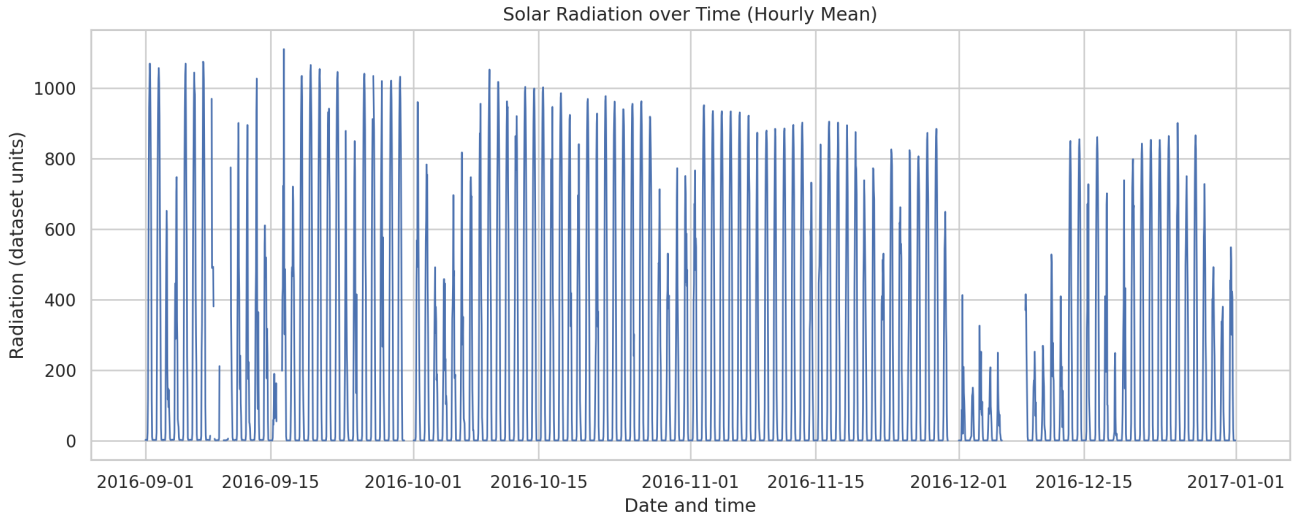


Figure 2. Relationships of radiation with temperature and humidity.

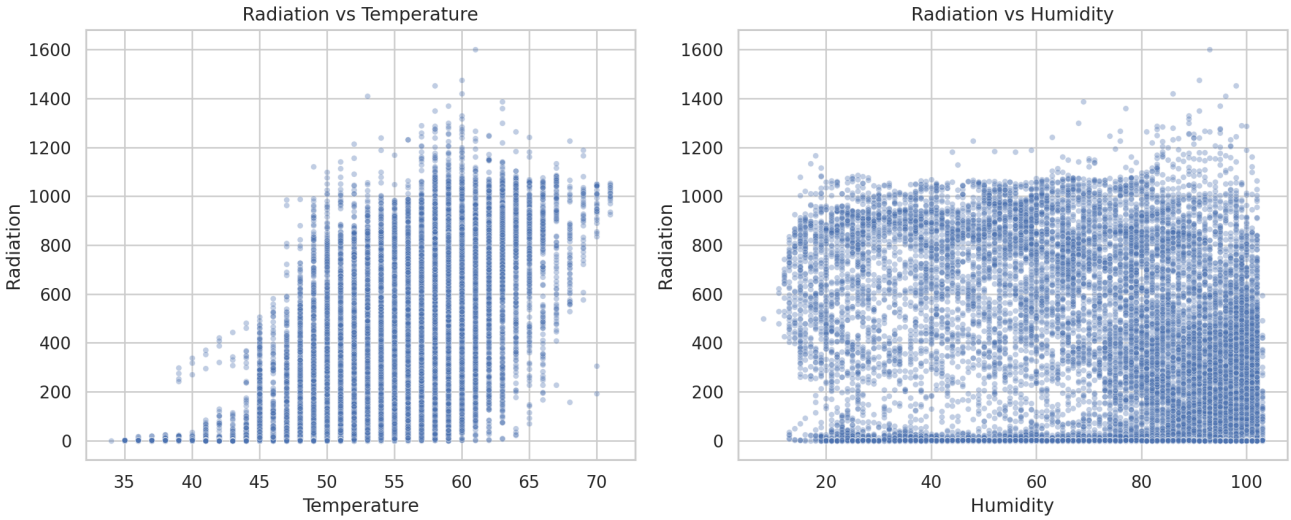


Figure 3. Correlation matrix for numerical environmental variables.

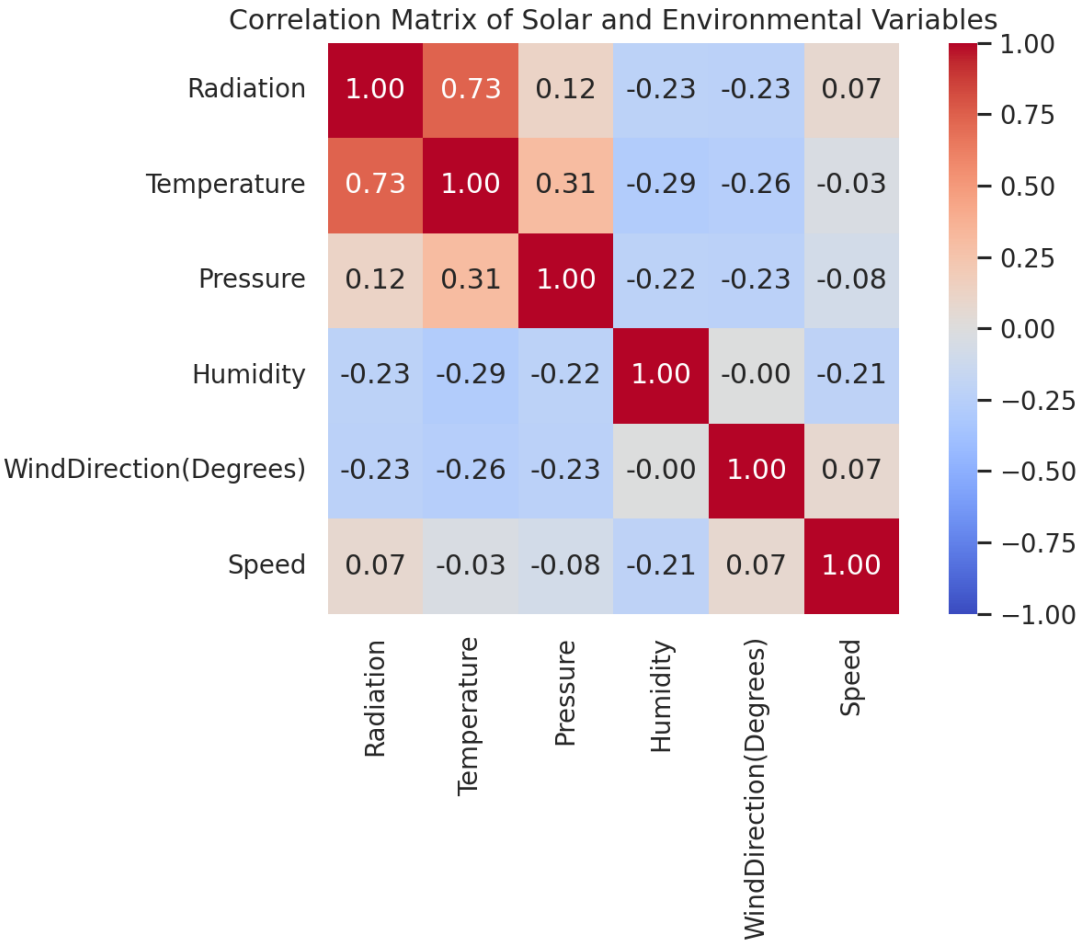


Figure 4. Bonus date-by-hour radiation heatmap.

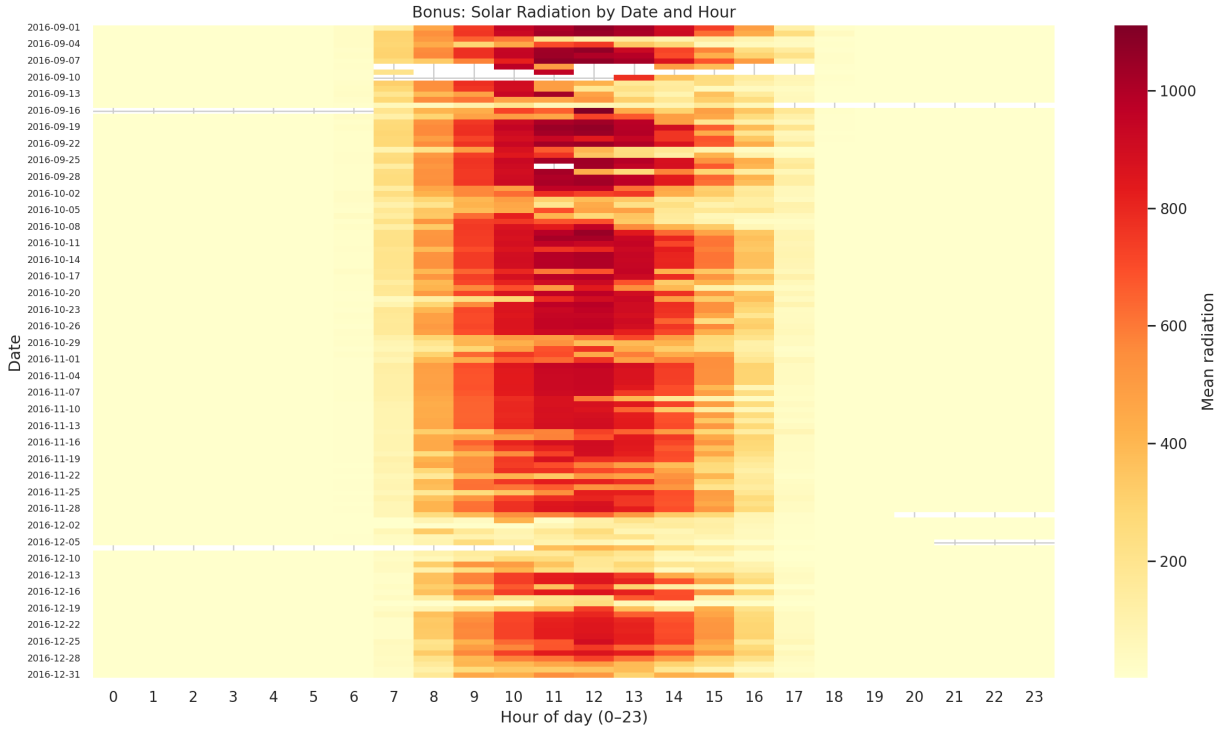
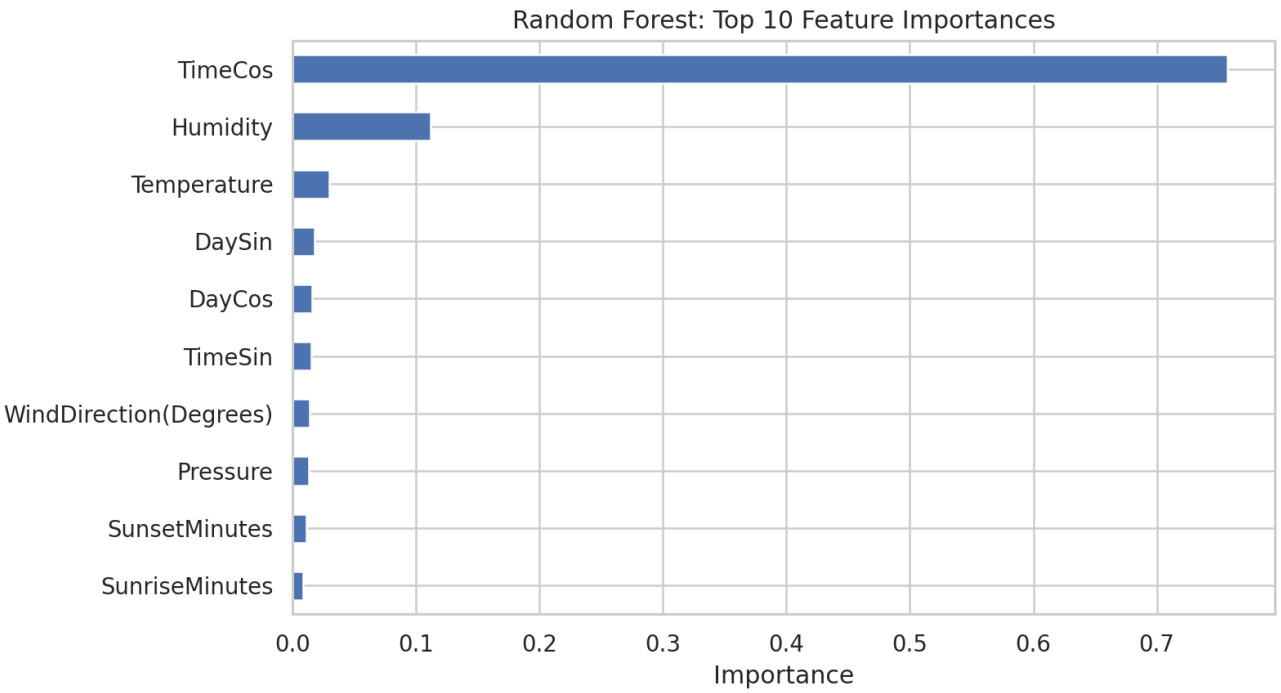


Figure 5. Most influential features in the fitted Random Forest model.



5. Interpretation and Limitations

Use the generated correlation values and plots to answer the PS2 inference questions. The peak hour is calculated from average radiation by clock hour, while the global peak refers to the single observation with the maximum recorded radiation; these are different statistics.

Potential outliers may reflect transient weather, cloud-edge effects, sensor noise, missing/incorrect readings, or other measurement issues. They should be described as possible explanations unless the dataset contains supporting metadata.

The Random Forest score is a baseline, not a guarantee of future forecasting performance. Because the split is random, nearby timestamps can appear in both train and test sets; for a stricter forecasting experiment, use a chronological split. Feature importance indicates how the fitted model used predictors, not whether a variable causes radiation changes.

Files produced by the script

- 01_radiation_over_time.png
- 02_environment_relationships.png
- 03_correlation_heatmap.png
- 04_bonus_date_hour_heatmap.png
- 05_model_feature_importance.png
- daily_solar_summary.csv
- model_feature_importance.csv
- analysis_summary.txt

6. Complete Python Code

The complete runnable source is also provided separately as **solar_ps2_analysis.py**. The listing below is the same code included in that file.

```
"""
PS2 — STAC WARS: Solar Radiation Analysis and Prediction
Dataset expected beside this script: olarPrediction.csv

Install dependencies:
    python -m pip install pandas numpy matplotlib seaborn scikit-learn

Run:
    python solar_ps2_analysis.py

Outputs are saved in the folder ./solar_ps2_outputs/
"""
from pathlib import Path
import warnings
warnings.filterwarnings("ignore", category=FutureWarning)

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_squared_error, r2_score

DATA_FILE = Path(__file__).resolve().parent / "olarPrediction.csv"
OUT_DIR = Path(__file__).resolve().parent / "solar_ps2_outputs"
OUT_DIR.mkdir(exist_ok=True)

def clock_to_minutes(series):
    """Convert clock strings such as '6:13:00' or '06:13:00' to minutes after midnight."""
    parsed = pd.to_datetime(series.astype(str), format="%H:%M:%S", errors="coerce")
    return parsed.dt.hour * 60 + parsed.dt.minute

def main():
    if not DATA_FILE.exists():
        raise FileNotFoundError(
            f"Could not find {DATA_FILE.name}. Put it in the same folder as this script."
        )

    df = pd.read_csv(DATA_FILE)
    required = [
        "Data", "Time", "Radiation", "Temperature", "Pressure", "Humidity",
        "WindDirection(Degrees)", "Speed", "TimeSunRise", "TimeSunSet"
    ]
    missing = [col for col in required if col not in df.columns]
    if missing:
        raise ValueError(f"Dataset is missing required columns: {missing}")

    # 'Data' contains a date/timestamp; use its calendar date and combine with 'Time'
    # because Time holds the actual observation clock time.
    date_part = pd.to_datetime(df["Data"], errors="coerce").dt.normalize()
    time_part = pd.to_timedelta(df["Time"].astype(str), errors="coerce")
    df["DateTime"] = date_part + time_part
    df["Date"] = df["DateTime"].dt.date
    df["Hour"] = df["DateTime"].dt.hour
    df["DayOfYear"] = df["DateTime"].dt.dayofyear

    numeric_cols = [
        "Radiation", "Temperature", "Pressure", "Humidity",
        "WindDirection(Degrees)", "Speed"
    ]
    for col in numeric_cols:
        df[col] = pd.to_numeric(df[col], errors="coerce")

    df = df.dropna(subset=["DateTime", "Radiation"]).copy()
    if df.empty:
        raise ValueError("No valid rows remain after parsing DateTime and Radiation.")

    # ------------------------- A. VISUALIZATION -------------------------
    sns.set_theme(style="whitegrid")
```

Complete Python Code (continued, lines 71–140)

```
071
072     # 1. Radiation over time: resample to hourly means to keep the plot readable.
073     time_series = df.set_index("DateTime")["Radiation"].sort_index()
074     hourly = time_series.resample("1h").mean()
075     fig, ax = plt.subplots(figsize=(12, 5))
076     ax.plot(hourly.index, hourly.values, linewidth=1.1)
077     ax.set_title("Solar Radiation over Time (Hourly Mean)")
078     ax.set_xlabel("Date and time")
079     ax.set_ylabel("Radiation (dataset units)")
080     fig.tight_layout()
081     fig.savefig(OUT_DIR / "01_radiation_over_time.png", dpi=180)
082     plt.close(fig)
083
084     # 2. Radiation versus temperature and humidity.
085     fig, axes = plt.subplots(1, 2, figsize=(12, 5))
086     sns.scatterplot(data=df, x="Temperature", y="Radiation", alpha=0.35, s=14, ax=axes[0])
087     axes[0].set_title("Radiation vs Temperature")
088     sns.scatterplot(data=df, x="Humidity", y="Radiation", alpha=0.35, s=14, ax=axes[1])
089     axes[1].set_title("Radiation vs Humidity")
090     fig.tight_layout()
091     fig.savefig(OUT_DIR / "02_environment_relationships.png", dpi=180)
092     plt.close(fig)
093
094     # 3. Correlation heatmap across numerical weather variables.
095     corr_cols = ["Radiation", "Temperature", "Pressure", "Humidity",
096                  "WindDirection(Degrees)", "Speed"]
097     corr = df[corr_cols].corr(numeric_only=True)
098     fig, ax = plt.subplots(figsize=(8, 6))
099     sns.heatmap(corr, annot=True, fmt=".2f", cmap="coolwarm", center=0,
100                 vmin=-1, vmax=1, square=True, ax=ax)
101     ax.set_title("Correlation Matrix of Solar and Environmental Variables")
102     fig.tight_layout()
103     fig.savefig(OUT_DIR / "03_correlation_heatmap.png", dpi=180)
104     plt.close(fig)
105
106     # Bonus: date x hour heatmap of mean radiation.
107     heat_data = df.pivot_table(index="Date", columns="Hour", values="Radiation", aggfunc="mean")
108     fig, ax = plt.subplots(figsize=(14, 8))
109     sns.heatmap(heat_data, cmap="YlOrRd", ax=ax, cbar_kws={"label": "Mean radiation"})
110     ax.set_title("Bonus: Solar Radiation by Date and Hour")
111     ax.set_xlabel("Hour of day (0–23)")
112     ax.set_ylabel("Date")
113     ax.tick_params(axis="y", labelsize=7)
114     fig.tight_layout()
115     fig.savefig(OUT_DIR / "04_bonus_date_hour_heatmap.png", dpi=180)
116     plt.close(fig)
117
118     # -------------------------- B. INFERENCES ---------------------------
119     hourly_mean = df.groupby("Hour")["Radiation"].mean().sort_index()
120     peak_hour = int(hourly_mean.idxmax())
121     correlations = df[corr_cols].corr(numeric_only=True)["Radiation"].drop("Radiation")
122     strongest_parameter = correlations.abs().idxmax()
123     strongest_corr = correlations[strongest_parameter]
124     temp_corr = correlations.get("Temperature", np.nan)
125     humidity_corr = correlations.get("Humidity", np.nan)
126
127     # -------------------------- C. DAILY REPORT -------------------------
128     daily = df.groupby("Date").agg(
129         DailyMaxRadiation=("Radiation", "max"),
130         DailyAverageRadiation=("Radiation", "mean"),
131         ObservationCount=("Radiation", "count")
132     ).reset_index()
133     daily = daily.sort_values("Date")
134     daily.to_csv(OUT_DIR / "daily_solar_summary.csv", index=False)
135
136     peak_idx = df["Radiation"].idxmax()
137     peak_row = df.loc[peak_idx]
138     peak_date = peak_row["Date"]
139     peak_time = peak_row["DateTime"]
140     sunrise = pd.to_datetime(str(peak_date) + " " + str(peak_row["TimeSunRise"]), errors="coerce")
```

Complete Python Code (continued, lines 141–210)

```
    sunset = pd.to_datetime(str(peak_date) + " " + str(peak_row["TimeSunSet"]), errors="coerce")
    after_sunrise = peak_time - sunrise if pd.notna(sunrise) else pd.NaT
    top5_days = daily.nlargest(5, "DailyAverageRadiation")

    # ------------------------- BONUS: ML PREDICTION --------------------
    # Cyclical time features preserve the wrap-around between 23:00 and 00:00.
    df["MinuteOfDay"] = df["Hour"] * 60 + df["DateTime"].dt.minute
    df["TimeSin"] = np.sin(2 * np.pi * df["MinuteOfDay"] / 1440)
    df["TimeCos"] = np.cos(2 * np.pi * df["MinuteOfDay"] / 1440)
    df["DaySin"] = np.sin(2 * np.pi * df["DayOfYear"] / 365.25)
    df["DayCos"] = np.cos(2 * np.pi * df["DayOfYear"] / 365.25)
    df["SunriseMinutes"] = clock_to_minutes(df["TimeSunRise"])
    df["SunsetMinutes"] = clock_to_minutes(df["TimeSunSet"])

    features = [
        "Temperature", "Pressure", "Humidity", "WindDirection(Degrees)", "Speed",
        "TimeSin", "TimeCos", "DaySin", "DayCos", "SunriseMinutes", "SunsetMinutes"
    ]
    model_df = df[features + ["Radiation"]].replace([np.inf, -np.inf], np.nan).dropna()
    X = model_df[features]
    y = model_df["Radiation"]
    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.20, random_state=42
    )
    model = RandomForestRegressor(
        n_estimators=150, random_state=42, n_jobs=-1, min_samples_leaf=2
    )
    model.fit(X_train, y_train)
    predictions = model.predict(X_test)
    rmse = float(np.sqrt(mean_squared_error(y_test, predictions)))
    r2 = float(r2_score(y_test, predictions))

    importance = pd.Series(model.feature_importances_, index=features).sort_values(ascending=False)
    importance.rename_axis("Feature").reset_index(name="Importance").to_csv(
        OUT_DIR / "model_feature_importance.csv", index=False
    )
    fig, ax = plt.subplots(figsize=(9, 5))
    importance.head(10).sort_values().plot(kind="barh", ax=ax)
    ax.set_title("Random Forest: Top 10 Feature Importances")
    ax.set_xlabel("Importance")
    fig.tight_layout()
    fig.savefig(OUT_DIR / "05_model_feature_importance.png", dpi=180)
    plt.close(fig)

    # Save a reproducible plain-text summary.
    with open(OUT_DIR / "analysis_summary.txt", "w", encoding="utf-8") as f:
        f.write("PS2 SOLAR RADIATION ANALYSIS — SUMMARY\n")
        f.write(f"Valid observations: {len(df):,}\n")
        f.write(f"Date range: {df['DateTime'].min()} to {df['DateTime'].max()}\n")
        f.write(f"Mean radiation: {df['Radiation'].mean():.4f}\n")
        f.write(f"Maximum radiation: {df['Radiation'].max():.4f}\n")
        f.write(f"Mean-radiation peak hour: {peak_hour:02d}:00\n")
        f.write(f"Strongest absolute linear correlation: {strongest_parameter} ({strongest_corr:.4f})\n")
        f.write(f"Temperature correlation: {temp_corr:.4f}\n")
        f.write(f"Humidity correlation: {humidity_corr:.4f}\n")
        f.write(f"Global peak date/time: {peak_date} / {peak_time}\n")
        f.write(f"Peak radiation: {peak_row['Radiation']:.4f}\n")
        f.write(f"Temperature at peak: {peak_row['Temperature']}\n")
        f.write(f"Humidity at peak: {peak_row['Humidity']}\n")
        f.write(f"Wind speed at peak: {peak_row['Speed']}\n")
        f.write(f"Sunrise: {sunrise}; sunset: {sunset}; peak after sunrise: {after_sunrise}\n")
        f.write(f"Random Forest RMSE: {rmse:.4f}\nRandom Forest R2: {r2:.4f}\n")
        f.write("\nTop 5 days by average radiation:\n")
        f.write(top5_days.to_string(index=False))
        f.write("\n\nTop model features:\n")
        f.write(importance.head(10).to_string())

    print("\n=== PS2 SOLAR ANALYSIS SUMMARY ===")
    print(f"Valid observations: {len(df):,}")
    print(f"Date range: {df['DateTime'].min()} to {df['DateTime'].max()}")
```

Complete Python Code (continued, lines 211–232)

```
211     print(f"Mean radiation: {df['Radiation'].mean():.4f}")
212     print(f"Maximum radiation: {df['Radiation'].max():.4f}")
213     print(f"Mean-radiation peak hour: {peak_hour:02d}:00")
214     print(f"Strongest absolute linear correlation: {strongest_parameter} ({strongest_corr:.4f})")
215     print(f"Temperature correlation: {temp_corr:.4f}")
216     print(f"Humidity correlation: {humidity_corr:.4f}")
217     print(f"Global peak date/time: {peak_date} / {peak_time}")
218     print(f"Peak radiation: {peak_row['Radiation']:.4f}")
219     print(f"Temperature at peak: {peak_row['Temperature']}")
220     print(f"Humidity at peak: {peak_row['Humidity']}")
221     print(f"Wind speed at peak: {peak_row['Speed']}")
222     print(f"Sunrise: {sunrise}; sunset: {sunset}; peak after sunrise: {after_sunrise}")
223     print("\nTop 5 days by average radiation:")
224     print(top5_days.to_string(index=False))
225     print(f"\nRandom Forest RMSE: {rmse:.4f}")
226     print(f"Random Forest R2: {r2:.4f}")
227     print("\nTop feature importances:")
228     print(importance.head(10).to_string())
229     print(f"\nAll charts and tables saved to: {OUT_DIR.resolve()}")
230
231 if __name__ == "__main__":
232     main()
```